

Glowly

AI beauty assistant — one photo → structured product intelligence

Final LLM Project

Problem

- People own 10–30 skincare products and don't know what's in them
- Can't tell what conflicts, what fits their skin, or how to layer them
- Reading INCI lists is unreadable; Googling each product is slow

User: anyone building a skincare routine who wants to understand their shelf

Solution

One photo →

- Brand · exact product · category
- Full ingredient list + active ingredients WITH their roles
- Skin-type fit · concerns targeted · how to use · safety flags

Editable, then saved to your collection.

Live Demo

Scan the Dr.Jart+ Dermaclear →

- reading label → identifying brand → scraping full INCI → structuring
- ~15s, then the filled confirmation screen

Result: Dr.Jart+ · cleanser · 61 ingredients · actives with roles

Backup: screenshots + curl output if Wi-Fi fails

Architecture

One request, traced end-to-end

iOS upload → FastAPI → LangGraph state machine → response

image_analysis → confidence_check →

■■ confidence < 0.75 → human confirmation (review screen)

■■ confidence ≥ 0.75 → save → recommendation generation

Services: Vision (Sonnet) · Scraper (INCIDecoder) · RAG (Qdrant) · MCP tools

Every LLM call traced in LangSmith

The 5-step AI pipeline

1. Vision — Claude Sonnet 4.6, single multimodal call (OCR + brand ID + JSON)
2. Vector cache — Qdrant, cosine ≥ 0.88 → instant repeat scans
3. Web scraping — INCIDecoder for the full INCI list
4. Embed & store — self-populating product database
5. Structured response — strict Pydantic → camelCase → Swift

→ Collapsing 3 lossy calls into 1 fixed our worst bug: 'Dr.Jart+' read as 'Mac'

Required modules — all load-bearing

- LangGraph workflow + conditional branch + human-in-the-loop
- Custom MCP server — 3 tools (analyze / categorize / recommend)
- Custom Skill — cosmetic-advisor with SKILL.md + triggers
- RAG — Qdrant + all-MiniLM-L6-v2, 63-doc knowledge base
- Web scraping — INCIDecoder (dynamic HTML)
- Multimodality — vision OCR is the core feature
- LangSmith tracing on every LLM call

Metrics & A/B experiment

We measured, then decided

Golden dataset: 41 examples · classification accuracy + LLM-as-judge

A/B — Haiku 4.5 vs Sonnet 4.6 (classification):

Pass rate: 92% vs 96% Avg score: 0.888 = 0.888 Cost: 1× vs ~8×

→ **Decision: Sonnet for vision, Haiku for text — same quality, 1/8 the cost**

→ **Failures are missing categories (hand cream, brow), NOT model errors**

Engineering trade-offs

- One combined vision call vs three — accuracy + cost, fixed misID
- Public /analyze inline vs full LangGraph — simpler client path
- Cache-first + scrape-fallback — fast, cheap, degrades gracefully
- Confidence threshold 0.75 — honest uncertainty → human review

Cost: ~1–3¢ per fresh scan; cached scans ≈ free

Conclusion & next steps

A complete, measured LLM product:

- vision + RAG + web scraping + LangGraph, observable in LangSmith
- evaluated on a 41-example golden dataset with an A/B experiment

Next: expand category taxonomy (from evals) · scrape ranking ·
model fallback · on-device routine recommendations

→ **One photo in, structured product intelligence out.**